

Lecture 1: Introduction to Operating Systems

What is an Operating System?

An operating system (OS) is system software that manages computer hardware and software resources and provides common services for computer programs. It acts as an intermediary between users and the computer hardware.

The OS performs several critical functions: process management, memory management, file system management, I/O device management, and security/protection.

Types of Operating Systems

Batch Operating Systems: Jobs are grouped and executed sequentially without user interaction during execution. Early mainframe computers used this approach.

Time-Sharing Systems: Multiple users share system resources simultaneously. The CPU switches between tasks rapidly, giving each user the illusion of dedicated access.

Real-Time Operating Systems (RTOS): Designed for applications requiring guaranteed response times. Used in embedded systems, medical devices, and industrial control.

Distributed Operating Systems: Manage a collection of independent computers that appear to users as a single coherent system.

Lecture 1 (cont.): OS Structure

Monolithic Kernel

In a monolithic kernel architecture, the entire OS runs as a single program in kernel mode. All OS services run in kernel space, providing fast system calls but making the system harder to maintain and debug.

Examples: Linux, traditional UNIX systems.

Microkernel Architecture

A microkernel keeps only the most essential services (IPC, basic scheduling, memory management) in kernel space. Other services (file systems, drivers, networking) run as user-space processes.

Advantages: better fault isolation, easier to extend. Disadvantages: potential performance overhead from IPC.

Examples: Minix, QNX, L4 family.

Hybrid Kernel

Combines aspects of monolithic and microkernel designs. Core services run in kernel mode for performance, but the system is modular enough to load/unload components dynamically.

Examples: Windows NT, macOS (XNU kernel).